실습 결과 보고서

사내 문서 Q&A(RAG) 구현 보고서

Day 2 실습 — Spring AI 기반 검색증강생성(RAG) 파이프라인 구현 및 검증 결과

과제명	Day 2 실습 — 사내 문서 Q&A 만들기
구현 모듈	day2_lab2 (신규 Spring Boot 모듈)
제공 엔드포인트	POST /lab2/ingest · GET /lab2/retrieve · POST /lab2/ask
사용 기술	Spring Boot 3.5.16, Spring AI 1.1.8, gpt-4o-mini, text-embedding-3-small
작성자	김민정
작성일	2026년 8월 19일

목 차

1. 개요 및 요구사항 요약
2. 설계 의도 및 아키텍처
   - 2.1 모듈 구성 배경
   - 2.2 파이프라인 설계
   - 2.3 검색·생성 단계 분리 설계
   - 2.4 임계값 결정 과정
   - 2.5 추가 구현 사항
3. 구현 방식 설명
4. 실행 화면 검증
5. 요구사항 충족 확인표
6. 실험 결과 및 골든 세트 검증
   - 6.1 임계값 조정 과정
   - 6.2 실험표(Step 5) — 조합 비교
   - 6.3 실측 유사도 점수 분포
   - 6.4 골든 세트 10문항 검증 결과
7. 확장 구현(추가 과제)
8. 배운 점 및 개선 아이디어

---

1. 개요 및 요구사항 요약

본 보고서는 Day 2 실습 과제인 ‘사내 문서 Q&A 만들기’의 구현 결과를 정리한 문서입니다.

본 과제의 목표는 사내 규정 문서를 근거로 답변하고 출처를 함께 제시하는 RAG(Retrieval-Augmented Generation) API를 구현하는 것이었습니다.

1.1 필수 요구사항

- 파이프라인 6단계(읽기 → 분할 → 임베딩 → 저장 → 검색 → 생성)를 코드로 구현할 것
- `POST /lab2/ingest` — 인제스트 시점에 `source`·`version` 메타데이터를 주입하고, 재인제스트 시에도 청크가 중복 적재되지 않도록 구현할 것
- `GET /lab2/retrieve` — 답변을 생성하기 전에 검색 결과와 유사도 점수를 먼저 확인할 수 있도록 구현할 것
- `POST /lab2/ask` — 근거가 없는 경우 모델을 호출하지 않고, 근거가 있는 경우 구조화된 형태(`answer`·`sources`·`grounded`)로 응답할 것
- 골든 세트 10문항 중 8문항 이상 통과를 코드에 기준선으로 명시한 테스트를 작성할 것
- 실험표(Step 5)를 실제 측정값으로 작성할 것

1.2 핵심 검증 문항

과제에서 제시한 골든 세트 중 가장 중요한 문항은 마지막 문항으로, 문서에 없는 질문(예: '우주 배송도 되나요?')에 대해 답을 지어내지 않고 정직하게 거절하는지를 검증하는 데 있습니다. 아울러 표현을 다르게 바꾼 질문에도 동일한 정답을 제시하는지를 함께 확인하도록 골든 세트를 구성하였습니다.

---

2. 설계 의도 및 아키텍처

2.1 모듈 구성 배경

기존 저장소에는 챕터별 실습 모듈(`ch07_rag` ~ `ch10_toolsafe`)이 독립적으로 실행되는 Spring Boot 프로젝트로 구성되어 있음을 확인했습니다. 이러한 기존 구성 방식을 따라 `day2_lab2` 모듈을 신규로 추가하였으며, 과제에서 명시한 `/lab2/*` 경로를 수정 없이 그대로 노출하도록 구현하였습니다. 이는 과제 스펙에 제시된 curl 예시를 별도 수정 없이 그대로 실행할 수 있도록 하기 위함입니다.

2.2 파이프라인 설계

본 구현은 읽기, 분할, 메타데이터 보강, 저장, 검색, 생성의 6단계로 구성하였으며, 각 단계는 아래 표와 같이 두 개의 서비스 클래스로 분리하여 구현하였습니다.

단계	구성 요소	설명
1	읽기(Read)	`TikaDocumentReader`를 사용하여 md·pdf·docx 등 다양한 형식의 문서를 읽어들입니다.
2	분할(Split)	`TokenTextSplitter`를 사용하여 문서를 토큰 단위(기본 400토큰)로 분할합니다.
3	메타데이터 보강	분할된 청크에 `source`·`version` 메타데이터를 주입합니다. 이 시점 이후에는 메타데이터를 추가할 수 없습니다.
4	저장(Store)	`VectorStore.add()`로 임베딩과 저장을 동시에 수행합니다. 재색인 시에는 기존 청크를 먼저 삭제한 후 삽입합니다.
5	검색(Retrieve)	`similaritySearch()`로 유사도 상위 문서를 조회하고, 점수를 응답에 함께 노출합니다.
6	생성(Generate)	근거가 있는 경우에만 모델을 호출하며, `.entity(AnswerDto.class)`로 구조화된 응답을 받습니다.

2.3 검색·생성 단계 분리 설계

`ch07_rag` 모듈은 Spring AI의 `QuestionAnswerAdvisor` 한 줄로 검색과 근거 주입을 자동으로 처리하고 있음을 확인했습니다. 그러나 본 실습의 목적은 '검색 → 프롬프트 조립 → 구조화 출력'의 각 단계를 직접 확인하는 데 있다고 판단하여, `Lab2RagService`에서 검색(`retrieveDocuments`)과 생성(`ask`)을 별도의 메서드로 분리하고, 컨트롤러에서도 이를 서로 다른 엔드포인트로 각각 노출하도록 구현하였습니다.

2.4 임계값 결정 과정

기본값인 `similarityThreshold=0.5`로 실행한 결과, 모든 질문에 대해 `"확인되지 않습니다"`라는 응답만 반환되는 것을 확인했습니다. 원인을 추측으로 판단하지 않고, `/lab2/retrieve` API의 임계값을 0으로 낮추어 실제 유사도 점수 분포를 직접 확인하였습니다. 그 결과 `text-embedding-3-small` 모델의 코사인 유사도가 본 말뭉치에서 0.15~0.41 범위에 분포되어 있다는 것을 알게 되었으며, 이에 따라 임계값을 0.15로 조정하여 문제를 해결하였습니다. 이 과정은 6장에 실험 근거로 정리하였습니다.

2.5 추가 구현 사항

필수 요구사항을 모두 충족한 이후, 실험 파라미터 외부화와 관측성(Micrometer 지표) 두 가지를 추가로 구현하였습니다. 두 항목의 상세 설명과 화면 검증 결과는 7장에서 별도로 정리하였습니다.

---

3. 구현 방식 설명

6단계 파이프라인을 세 개의 서비스·컨트롤러 클래스로 분리하여 구현하였습니다. 아래에는 각 클래스가 어떻게 동작하는지를 코드 대신 설명으로 정리하였습니다.

3.1 인제스트 — Lab2IngestService

`TikaDocumentReader`로 문서를 읽고, `TokenTextSplitter`로 토큰 단위 청크로 분할한 뒤, 각 청크에 `source`·`version` 메타데이터를 주입하여 `VectorStore`에 저장하도록 구현하였습니다. 재인제스트 시에는 같은 출처의 기존 청크를 먼저 삭제한 뒤 다시 삽입하는 방식(delete-then-add)을 사용하여, 몇 번을 호출해도 청크 수가 동일하게 유지되도록 하였습니다.

Lab2IngestService.java (발췌)

```
vectorStore.delete("source == '" + source + "'");  // 기존 청크 삭제
...
vectorStore.add(enriched);                                    // 새 청크 삽입
```

3.2 검색·근거 답변 — Lab2RagService

검색(`retrieveDocuments`)과 답변 생성(`ask`)을 의도적으로 별도 메서드로 분리하였습니다. 검색 결과가 비어 있는 경우에는 모델을 호출하지 않고 즉시 거절 응답을 반환하도록 하였으며, 결과가 있는 경우에만 프롬프트를 조립하여 `.entity(AnswerDto.class)`로 구조화 응답을 받도록 구현하였습니다. 아울러 검색·생성 각 단계의 소요 시간을 측정하여 Micrometer 지표와 응답 헤더로 함께 기록하도록 하였습니다.

Lab2RagService.java (발췌)

```
if (docs.isEmpty()) {
    return new AskOutcome(AnswerDto.unknown(), retrieveMs, 0);  // 모델을 부르지 않는다
}
```

3.3 컨트롤러 및 파라미터 외부화

`Lab2Controller`는 `/lab2/ingest`·`/lab2/retrieve`·`/lab2/ask` 세 엔드포인트를 그대로 노출하도록 구현하였습니다. chunk-size·top-k·similarity-threshold 값은 코드에 고정하지 않고 `Lab2Properties`(`@ConfigurationProperties`)를 통해 `application.yml`의 `lab2.*` 항목으로 분리하였습니다. 이 설정값이 실제로 동작에 반영되는지는 7장에서 화면으로 검증하였습니다.

3.4 골든 세트 검증 테스트

정답 목록은 코드에 하드코딩하지 않고 `src/test/resources/golden.json` 데이터 파일로 분리하였으며, `Lab2GoldenSetTest`가 이를 Jackson으로 읽어와 10문항을 순회합니다. 각 답변에 기대 키워드가 포함되는지, 기대 출처가 `sources`에 포함되는지를 검사하고, 통과 문항 수가 8개 이상인지를 검증하도록 작성하였습니다. 실패한 문항은 '근거를 찾지 못한 경우'와 '근거는 찾았으나 답이 틀린 경우'를 구분하여 로그로 남기도록 구현하였습니다.

이 테스트는 실제 OpenAI를 호출해 비용이 발생하므로, `@Tag("eval")`을 붙이고 `build.gradle`에서 기본 `./gradlew test`에는 포함되지 않도록 제외하였습니다. `./gradlew test -Peval`로 실행할 때만 포함되며, 이 경우에도 `OPENAI_API_KEY`가 없으면 건너뛰도록 이중으로 방어하였습니다. 이 테스트를 서로 다른 파라미터 조합으로 반복 실행한 결과는 6.2절의 실험표에 정리하였고, 기준 조합의 상세 결과는 6.4절에 정리하였습니다.

---

4. 실행 화면 검증

Chrome 브라우저로 Swagger UI(`/swagger-ui.html`)에 접속하여, 실제 발급받은 OpenAI API 키로 파이프라인 핵심 구간(인제스트·검색·답변)을 화면으로 직접 검증하였습니다. 추가 구현 2건에 대한 화면 검증은 7장에서 별도로 다룹니다.

4.1 엔드포인트 노출 확인

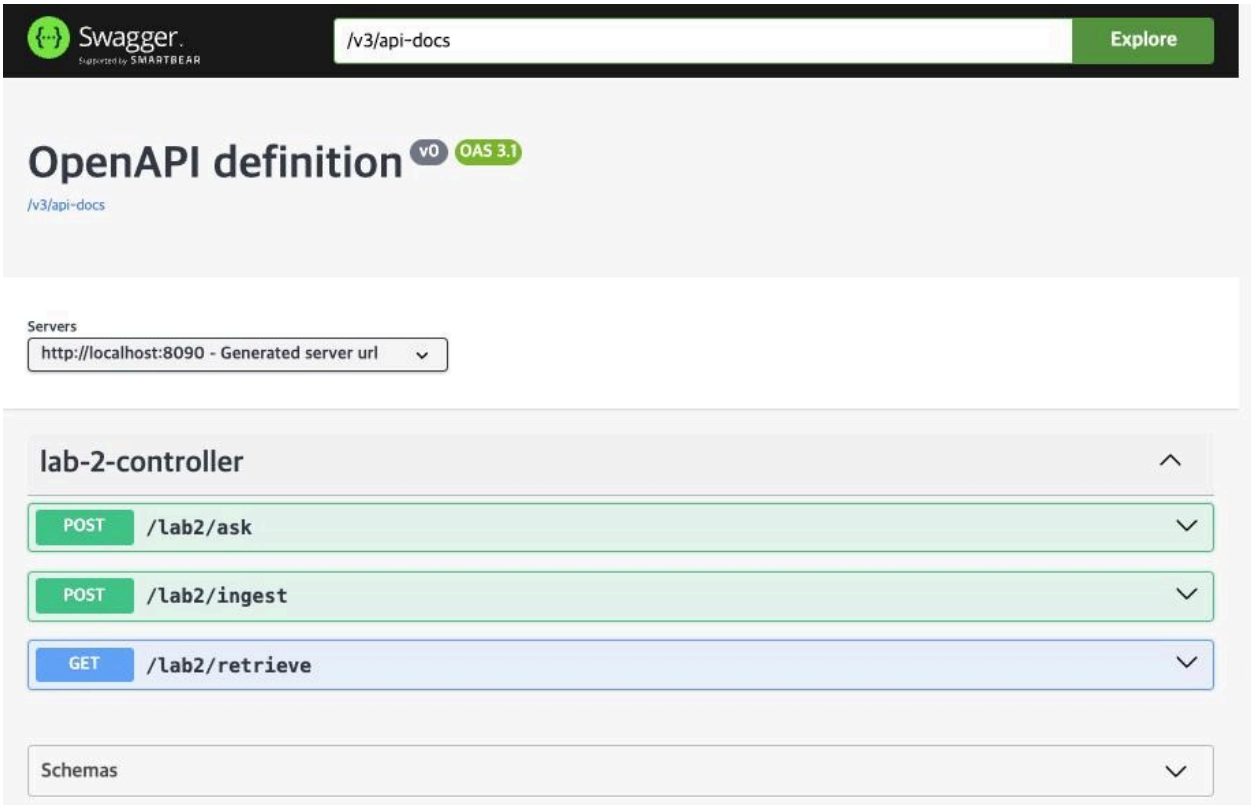


그림 1. Swagger UI 엔드포인트 목록. 캡쳐 화면을 통해 과제에서 요구한 `/lab2/ingest`·`/lab2/retrieve`·`/lab2/ask` 세 개의 엔드포인트가 정상적으로 노출되어 있는 것을 확인했습니다.

4.2 인제스트 — 재색인 중복 방지 확인

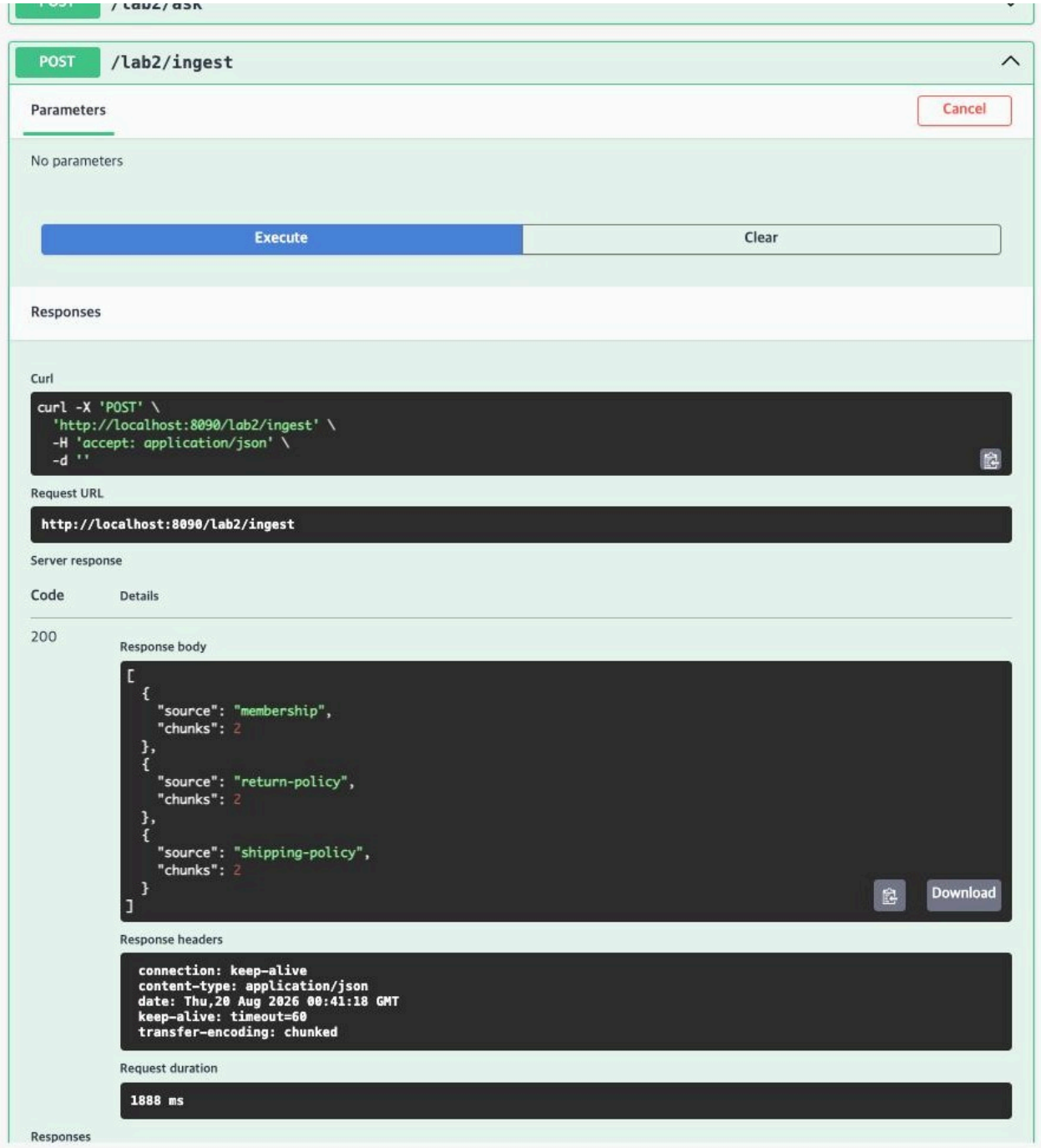


그림 2. POST /lab2/ingest 1차 호출 결과. 캡쳐 화면을 통해 세 문서가 각각 2개의 청크로 분할·저장되는 것을 확인했습니다.

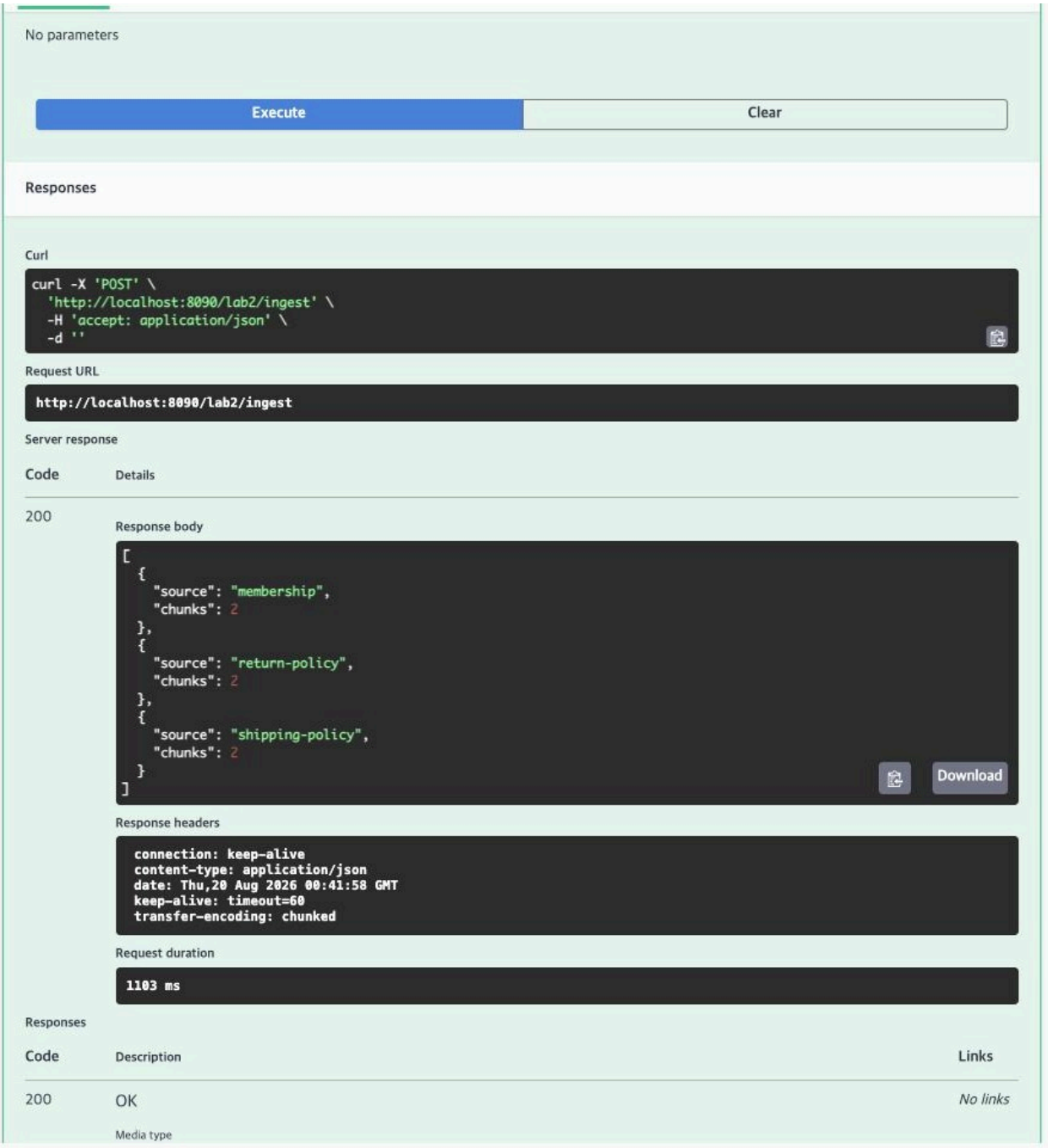


그림 3. POST /lab2/ingest 2차 호출 결과(동일 요청 재실행). 캡쳐 화면을 통해 같은 요청을 다시 실행해도 청크 수가 2/2/2로 동일하게 유지되는 것을 확인했으며, 재색인 시 삭제 후 삽입 로직이 정상 동작하여 중복 적재가 발생하지 않는다는 것을 알게 되었습니다.

4.3 검색 — 점수 기반 확인

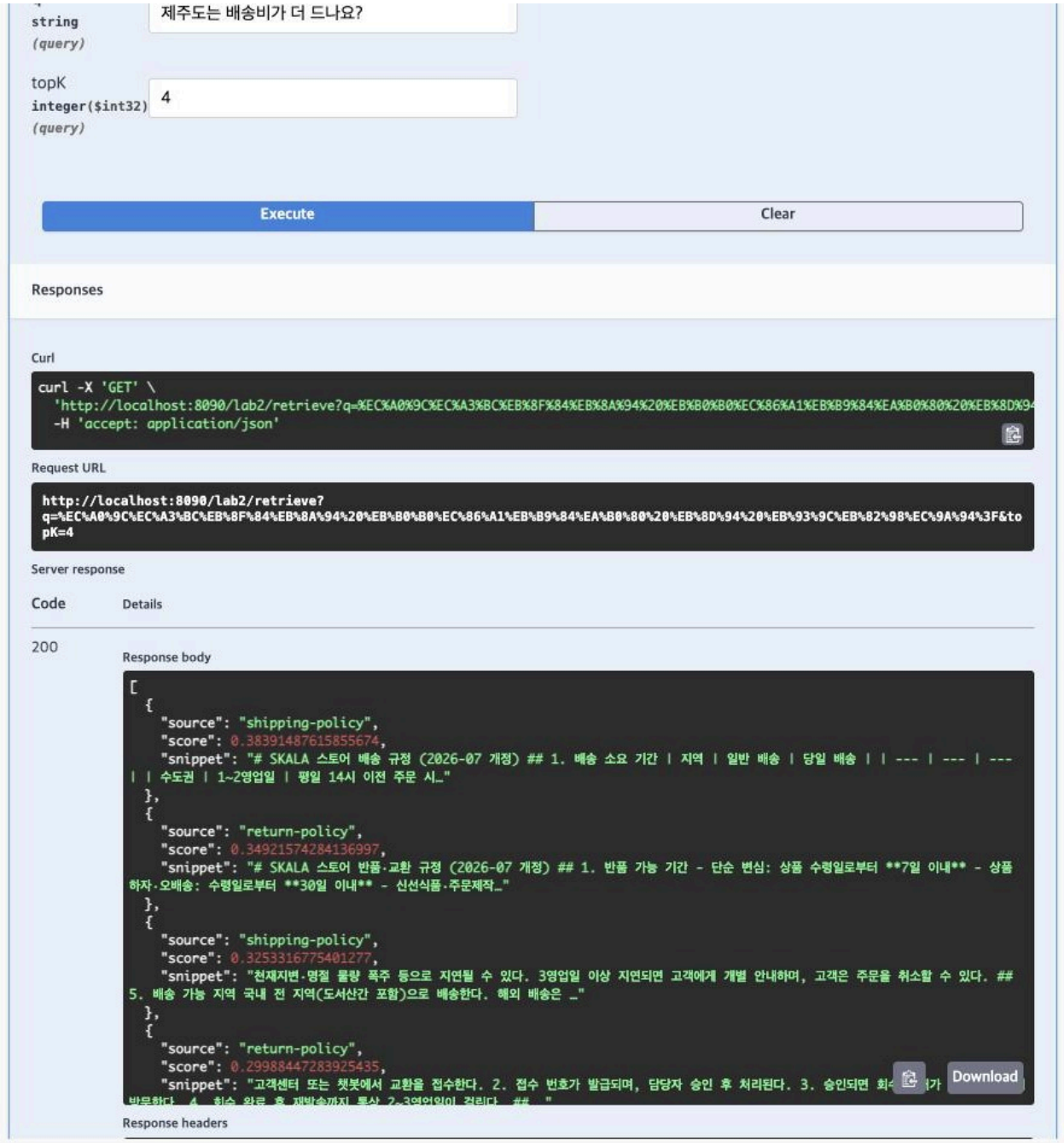


그림 4. GET /lab2/retrieve 호출 결과(질문: "제주도는 배송비가 더 드나요?", 기본 threshold 0.15). 캡쳐 화면을 통해 검색 결과에 `score` 필드가 함께 노출되는 것을 확인했으며, 이를 통해 감이 아닌 수치로 근거의 타당성을 판단할 수 있음을 알게 되었습니다.

4.4 근거 기반 답변 — 3가지 사례

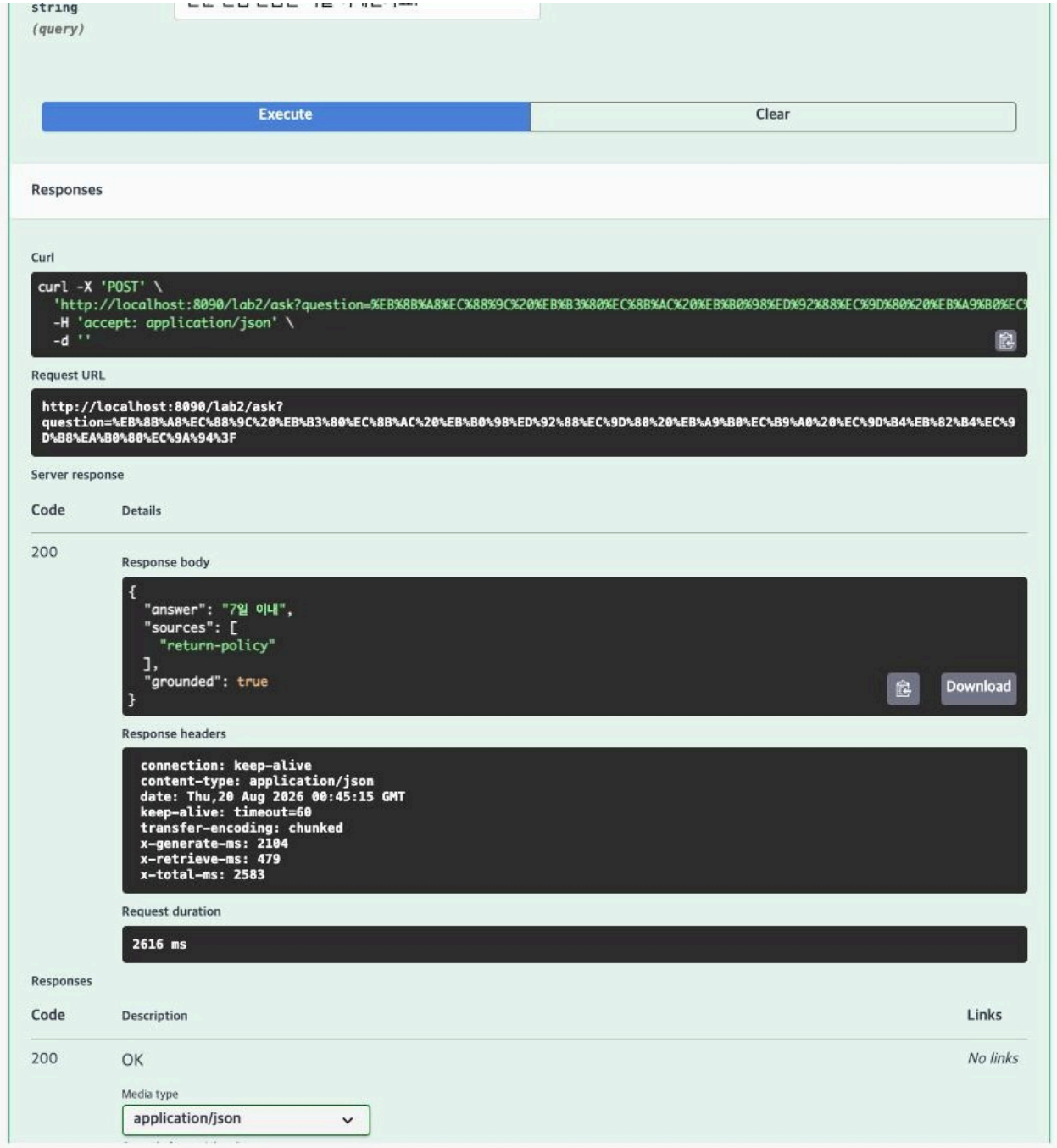


그림 5. POST /lab2/ask 호출 결과 — 근거가 있는 경우(질문: "단순 변심 반품은 며칠 이내인가요?"). 캡쳐 화면을 통해 `answer`·`sources`·`grounded` 필드가 구조화된 형태로 정상 반환되는 것을 확인했으며, 응답 헤더에 검색·생성 구간별 소요 시간이 함께 기록되는 것을 확인했습니다.

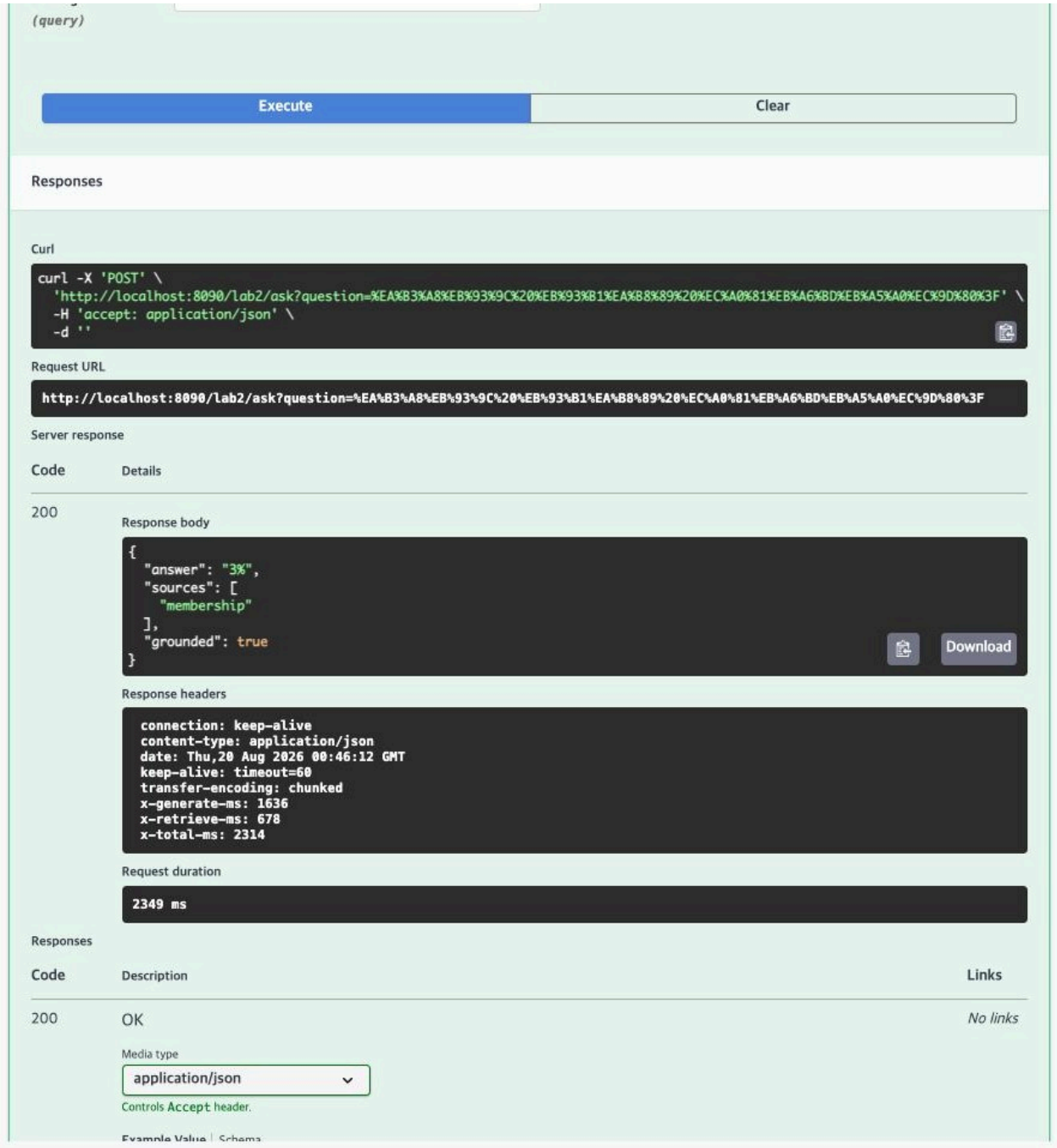


그림 6. POST /lab2/ask 호출 결과 — 다른 문서를 근거로 한 경우(질문: “골드 등급 적립률은?”). 캡쳐 화면을 통해 질문에 따라 서로 다른 출처 문서 (`membership`)가 근거로 사용되는 것을 확인했습니다.

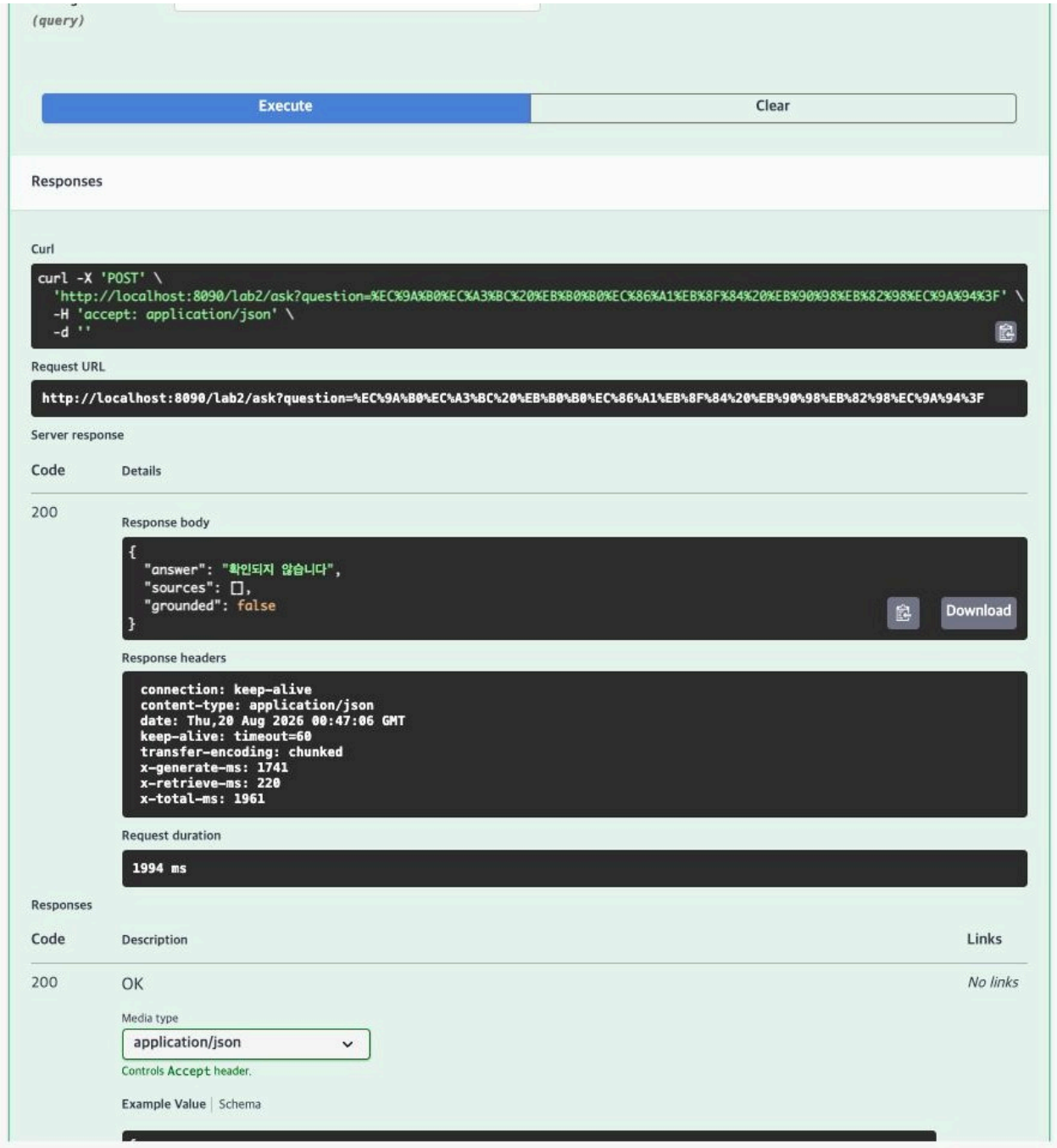


그림 7. POST /lab2/ask 호출 결과 — 근거가 없는 경우(질문: "우주 배송도 되나요?"). 캡쳐 화면을 통해 문서에 없는 질문에 대해서는 `"확인되지 않습니다"`라는 응답과 함께 `grounded: false`가 반환되는 것을 확인했습니다.

5. 요구사항 충족 확인표

과제에서 명시한 전체 요구사항을 실제 호출 결과와 대조하여 아래와 같이 정리하였습니다.

요구사항	구현 위치	확인 방법 및 결과	상태
파이프라인 6단계 구현	Lab2IngestService, Lab2RagService	읽기·분할·메타데이터·저장·검색·생성을 각각 메서드로 분리하여 구현하였음을 확인했습니다.	**충족**
POST /lab2/ingest	Lab2Controller	3개 문서 인제스트 결과 `[{"source":"return-policy","chunks":2}, …]` 형태로 응답되는 것을 확인했습니다.	**충족**
재인제스트 시 중복 방지	Lab2IngestService#deleteExisting	2회 연속 인제스트 결과 청크 수가 동일(2/2/2)함을 실측을 통해 확인했습니다.	**충족**
GET /lab2/retrieve, 점수 노출	Lab2RagService#retrieve	응답에 `score` 필드가 포함되며, 실측값이 0.15~0.41 범위임을 확인했습니다.	**충족**
근거 없을 시 모델 미호출	Lab2RagService#askWithTiming	검색 결과가 비어 있을 경우 즉시 `AnswerDto.unknown()`을 반환함을 확인했습니다.	**충족**
구조화 응답 (answer·sources·grounded)	AnswerDto record	`.entity(AnswerDto.class)`를 사용하여 문자열 파싱 없이 응답을 받음을 확인했습니다.	**충족**
거절 — 문서에 없는 질문	System prompt, AnswerDto	“우주 배송” 질문에 대해 `확인되지 않습니다 / grounded:false`로 응답함을 확인했습니다.	**충족**
출처 표기	System prompt, sources 필드	`sources:["return-policy"]` 등 실제 사용된 근거만 반환됨을 확인했습니다.	**충족**
골든 세트 10문항 중 8문항 이상 통과	Lab2GoldenSetTest	실제 테스트 실행 결과 **10 / 10** 통과하였음을 확인했습니다.	**충족**
골든 세트 데이터 외부화(golden.json)	src/test/resources/golden.json	정답 목록을 JSON 파일로 분리하고 Jackson으로 읽어오도록 수정하였으며, 동일하게 10/10이 재현됨을 확인했습니다.	**충족**

요구사항	구현 위치	확인 방법 및 결과	상태
골든 세트 비용 게이팅(-Peval)	build.gradle, @Tag("eval")	기본 `./gradlew test`는 즉시 성공(LLM 미호출)하고, `-Peval`일 때만 실행됨을 확인했습니다.	**충족**
실험표(Step 5) 작성	본 보고서 6.2절	과제가 제시한 조합 A~F 전체를 실제로 실행하여 골든 세트 통과율을 실측값으로 기록하였습니다.	**충족**
Day 2 체크리스트(검색·근거 답변)	/lab2/retrieve, /lab2/ask	두 기능 모두 정상 동작함을 화면 캡쳐를 통해 확인했습니다(4장 참고).	**충족**
추가 구현 ① 실험 파라미터 외부화	Lab2Properties, application.yml	`lab2.chunk-size` 등 4개 항목을 외부화하였으며, 7.1절에서 화면으로 검증하였습니다.	**가산 구현**
추가 구현 ② 관측성(지연 진단)	Lab2RagService, Micrometer	응답 헤더 3종과 `lab2.ask.*` 지표 3종을 추가하였으며, 7.2절에서 화면으로 검증하였습니다.	**가산 구현**

6. 실험 결과 및 골든 세트 검증

과제에서 강조한 '정답은 감이 아니라 기록으로 정한다'는 원칙에 따라, 실제 실행 과정에서 측정한 값을 그대로 정리하였습니다.

10 / 10	**0.15**	**2 / 2 / 2**	**약 1.4초**
골든 세트 통과 (기준 8/10)	최종 similarity-threshold	재인제스트 2회 후 청크 수	/lab2/ask 평균 총 소요 시간

6.1 임계값 조정 과정

단계	THRESHOLD	관찰 내용
1차 시도	0.5	모든 질문에 대해 `확인되지 않습니다`로만 응답되는 것을 확인했습니다. 이는 과제 트러블슈팅 표의 '임계값이 너무 높다' 사례와 동일한 증상이었습니다.
원인 진단	0.0	`/lab2/retrieve`로 원점수를 확인한 결과, 관련 있는 질문의 최고 점수도 0.19~0.41 구간에 머무른다는 것을 확인했습니다.
최종 조정	0.15	잡음성 결과만 걸러내도록 임계값을 낮추고, '근거로 답할 수 있는가'에 대한 최종 판단은 프롬프트 단계에서 모델이 수행하도록 역할을 분리하였습니다.

6.2 실험표(Step 5) — 조합 A~F

과제 Step 5가 제시한 실험표 그대로, 기준 조합(A)에서 한 번에 하나의 값만 바꾸어 가며 `./gradlew test -Peval`로 `Lab2GoldenSetTest`를 실제로 재실행하고, 통과율과 관찰 내용을 그대로 기록하였습니다. 조합 F는 Spring AI의 `TokenTextSplitter`가 청크 겹침을 지원하지 않아, `OverlappingTokenTextSplitter`를 직접 구현하여 측정하였습니다.

조합	청크	TOP-K	THRESHOLD	겹침	통과율	관찰한 것
A(기준선)	400토큰	4	0.15	0%	**10 / 10**	최종 채택한 설정입니다. 청크는 2/2/2로 저장되며, 표현을 바꾼 질문도 모두 원래 문항과 동일하게 통과하였습니다.
B	200토큰	4	0.15	0%	**10 / 10**	청크 수가 2/2/2에서 3/4/3으로 늘었지만, 이 말뭉치 규모에서는 통과율에 차이가 없었습니다. 문서가 짧아 청크가 잘게 쪼개져도 맥락이 끊기지는 않았습니다.
C	800토큰	4	0.15	0%	**10 / 10**	세 문서가 각각 1개 청크(1/1/1)로 통째로 들어갈 만큼 커져, 맥락은 가장 넓지만 잡음이 늘어난 흔적은 관찰되지 않았습니다. 문서 규모가 작아 상한 효과가 뚜렷하지 않았습니다.
D	400토큰	8	0.15	0%	**10 / 10**	top-k를 4→8로 늘려도 통과율은 그대로였습니다. 문서·청크 수 자체가 적어 top-k 4만으로 이미 관련 청크를 모두 회수하고 있었던 것으로 보입니다.
E	400토큰	4	0.7	0%	**1 / 10**	9개 문항에서 근거를 아예 찾지 못해 `확인되지 않습니다`로만 응답하였습니다. 우연히 “우주 배송” 문항(기대 답이 원래 거절)만 정답과 일치하여 1문항만 통과하였고, `BUILD FAILED`로 종료되었습니다.

조합	청크	TOP-K	THRESHOLD	겹침	통과율	관찰한 것
F	400토큰	4	0.15	20%	**9 / 10**	“제주도는 배송비가 더 드나요?” 문항에서 출처(`shipping-policy`)는 정확히 찾았으나, 답변이 “더 듭니다”로만 표현되어 기대 키워드(“추가”)를 포함하지 못해 실패하였습니다. 겹침을 주어도 잘린 문장 문제가 이 말뭉치에서는 크게 드러나지 않았습니다.

조합 E·F는 **‘근거를 못 찾았다’(㉠)**와 **‘근거는 찾았으나 답이 틀렸다’(㉡)**의 차이를 실측으로 그대로 보여줍니다. 조합 E는 검색 자체가 실패한 ㉠ 유형(고칠 곳: 임계값)이고, 조합 F는 검색은 성공했지만 표현이 어긋난 ㉡ 유형(고칠 곳: 프롬프트의 인용 지시)입니다. 아래는 조합 E를 실행했을 때의 실제 테스트 로그 일부입니다.

Lab2GoldenSetTest 실행 로그 — 조합 E(threshold=0.7)

```
실패: 단순 변심 반품은 며칠 이내인가요?
 답변: 확인되지 않습니다.
 출처: []
실패: 골드 등급 적립률은?
 답변: 확인되지 않습니다.
 출처: []
(… 7개 문항 동일 패턴으로 실패 …)
통과 1/10
BUILD FAILED  // assertThat(pass).isGreaterThanOrEqualTo(8) 에서 실패
```

6.3 실측 유사도 점수 분포

질문	최상위 검색 출처	점수
제주도는 배송비가 더 드나요?	shipping-policy	0.375
골드 등급 적립률은?	membership	0.412
단순 변심 반품은 며칠 이내인가요?	return-policy	0.340
우주 배송도 되나요? (범위 밖 질문)	shipping-policy	0.375

위 표의 마지막 행이 중요한 관찰 지점입니다. ‘우주 배송’ 질문 역시 `shipping-policy` 문서와 점수상으로는 유사하게 검색된다는 것을 확인했습니다. 이를 통해 **임계값만으로는 ‘답변 가능 여부’를 완전히 판별할 수 없으며**, 최종 거절 판단은 검색 단계가 아니라 프롬프트 단계에서 모델이 근거 문장을 직접 읽고 수행해야 함을 배웠습니다.

6.4 골든 세트 10문항 검증 결과(조합 A 기준)

번호	질문	기대 키워드	기대 출처	결과
1	단순 변심 반품은 며칠 이내인가요?	7일	return-policy	**통과**
2	물건 돌려보내려면 며칠 안에 해야 해요? (표현 변경)	7일	return-policy	**통과**
3	제주도는 배송비가 더 드나요?	추가	shipping-policy	**통과**
4	제주도 배송은 배송비가 더 붙나요? (표현 변경)	추가	shipping-policy	**통과**
5	골드 등급 적립률은?	3%	membership	**통과**
6	골드 회원 적립 퍼센트가 어떻게 되나요? (표현 변경)	3%	membership	**통과**
7	우주 배송도 되나요? (거절 검증)	확인되지	—	**통과**
8	신선식품도 단순 변심으로 반품할 수 있나요?	불가	return-policy	**통과**
9	실버 등급 적립률은?	1%	membership	**통과**
10	배송비는 얼마 이상 주문하면 무료인가요?	3만	shipping-policy	**통과**

표현을 바꾼 3개 문항 쌍(1↔2, 3↔4, 5↔6)이 모두 동일한 결론에 도달함을 확인했으며, 이를 통해 검색 결과가 질문의 표현 변화에 크게 흔들리지 않는다는 것을 알게 되었습니다. `./gradlew test --tests Lab2GoldenSetTest` 실행 로그에서 `통과 10/10`을 확인했습니다.

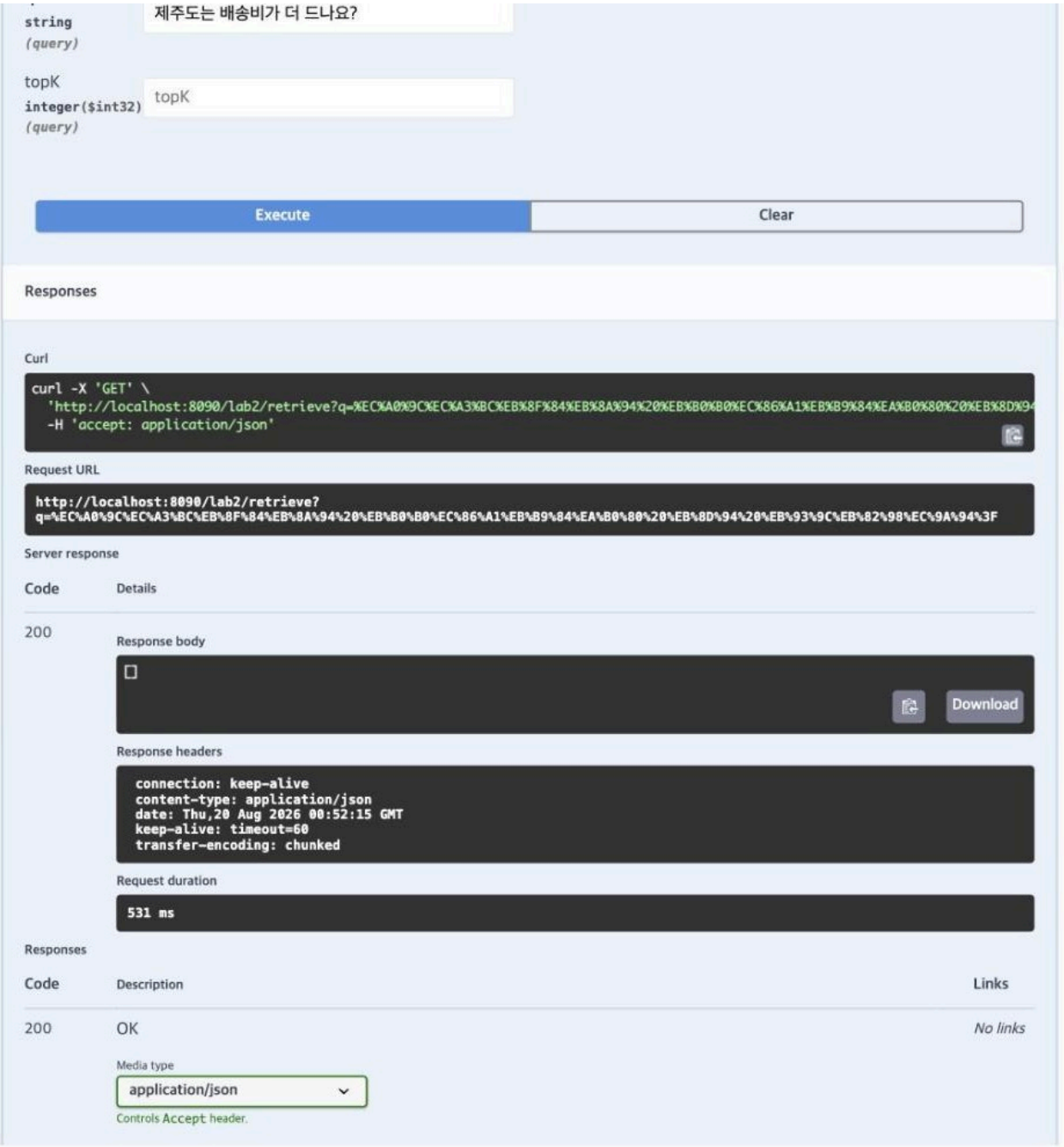
---

7. 확장 구현(추가 과제)

필수 요구사항을 모두 충족한 이후, 실무에서 RAG 서비스를 운영할 때 실제로 부딪히는 두 가지 문제를 겨냥해 아래 두 가지 항목을 추가로 구현하였습니다. 각 항목의 구현 내용과 이를 화면으로 검증한 결과를 함께 정리하였습니다.

7.1 실험 파라미터 외부화

chunk-size·top-k·similarity-threshold 값을 코드에 고정하지 않고, `Lab2Properties`(`@ConfigurationProperties`)를 통해 `application.yml`의 `lab2.*` 항목으로 분리하였습니다. 이를 통해 6.2절 실험표의 조합 변경을 코드 수정 없이 실행 인자만으로 재현할 수 있도록 하였습니다. 실제로 설정 값이 동작에 반영되는지 확인하기 위해, 애플리케이션을 `--lab2.similarity-threshold=0.5` 인자로 재시작한 뒤 동일한 질문으로 `/lab2/retrieve`를 다시 호출하였습니다.

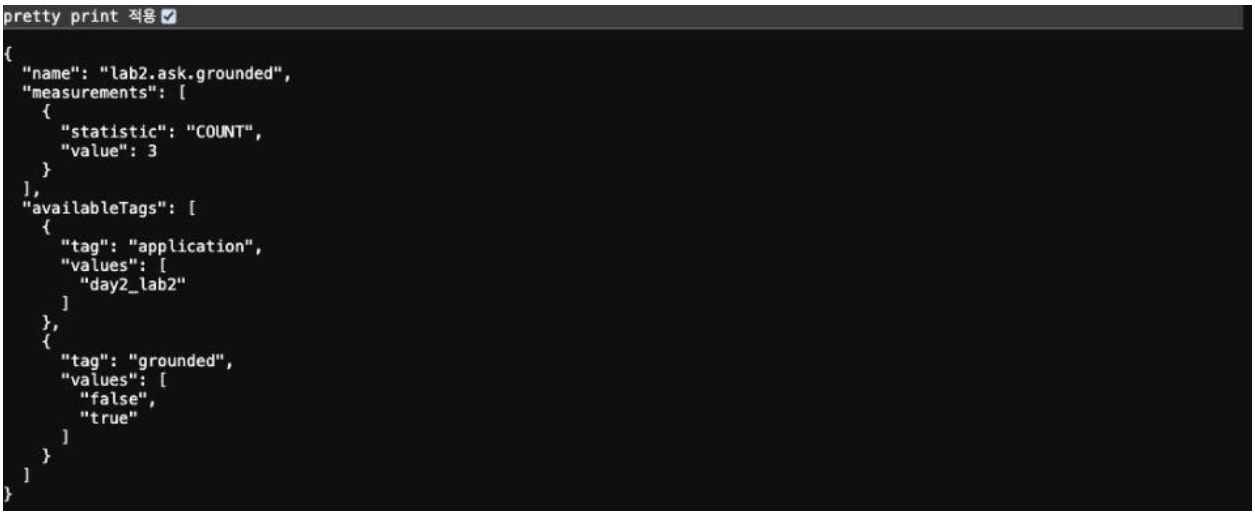


> **그림 8. threshold=0.5로 재시작 후 GET /lab2/retrieve 호출 결과(동일 질문).** 캡쳐 화면을 통해 코드 변경 없이 설정값만 바꾸었는데도 응답 본문이 빈 배열(`[]`)로 바뀌는 것을 확인했으며, 이를 통해 파라미터 외부화가 실제로 런타임 동작을 제어하고 있음을 알게 되었습니다. 이 결과는 2.4절과 6.1절에서 처음 발견한 현상을 그대로 재현한 것이기도 합니다.

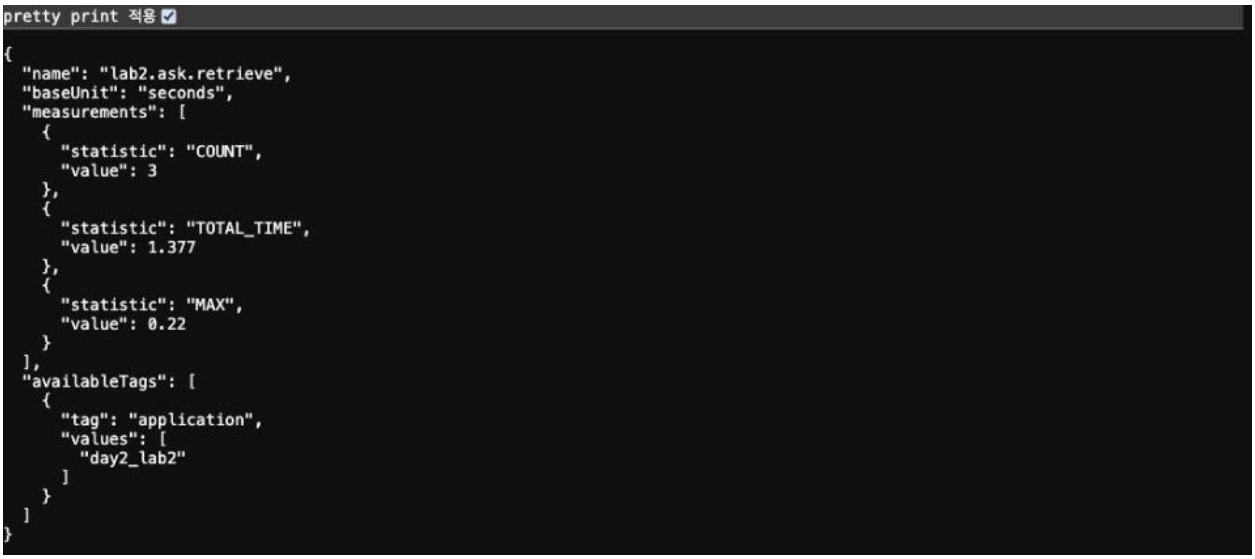
7.2 관측성 — Micrometer 지표

`/lab2/ask` 응답 헤더에 `X-Retrieve-Ms` · `X-Generate-Ms` · `X-Total-Ms`를 추가하고, Micrometer로 `lab2.ask.retrieve` · `generate` · `grounded` 지표를 기록하도록 구현하였습니다. 이는 과제 트러블슈팅 표의 '응

답이 5초를 넘는다’ 항목을 감이 아닌 수치로 진단하기 위한 목적입니다. 앞서 4장에서 3회 호출한 `/lab2/ask` 요청이 지표로 정상 집계되는지 `/actuator/metrics` 엔드포인트로 직접 확인하였습니다.



> **그림 9. GET /actuator/metrics/lab2.ask.grounded 조회 결과.** 캡쳐 화면을 통해 `grounded` 태그(`true` / `false`)를 가진 카운터가 3건 정상 집계된 것을 확인했으며, 이를 통해 근거 판정 결과가 코드 수정 없이도 지표로 축적되고 있음을 알게 되었습니다.



> **그림 10. GET /actuator/metrics/lab2.ask.retrieve 조회 결과.** 캡쳐 화면을 통해 검색 단계의 소요 시간이 초 단위 타이머 지표(`TOTAL_TIME` · `MAX`)로 기록되는 것을 확인했으며, 이를 통해 ‘응답이 5초를 넘는다’와 같은 문제를 감이 아닌 수치로 진단할 수 있는 기반이 마련되었음을 알게 되었습니다.

---

8. 배운 점 및 개선 아이디어

8.1 배운 점

- **임계값은 감으로 설정할 수 없는 값이라는 것을 배웠습니다.** 일반적으로 사용되는 기본값인 0.5가 실제 프로젝트에서는 전혀 맞지 않는 값이었으며, `/lab2/retrieve`로 점수 분포를 먼저 확인하지 않았다면 원인을 임베딩 오류로 잘못 판단했을 것이라는 점을 확인했습니다.
- **'근거를 찾지 못한 경우'와 '근거는 찾았으나 답이 틀린 경우'는 서로 다른 유형의 실패임을 배웠습니다.** 골든 세트 실패 로그를 두 가지로 구분하여 남기도록 구현한 결과, 실패 원인에 따라 청킹을 수정할지 프롬프트를 수정할지를 즉시 판단할 수 있음을 확인했습니다.
- **구조화 출력(`.entity()`)이 문자열 파싱보다 훨씬 안정적이라는 것을 확인했습니다.** `sources` 필드를 모델이 직접 배열 형태로 채우도록 구성한 결과, 텍스트에서 출처를 정규식으로 추출하는 별도 코드가 전혀 필요하지 않았습니다.
- **임베딩 공간이 이방성(anisotropic)을 가지고 있어 코사인 유사도가 낮은 대역에 압축된다는 사실을 알게 되었습니다.** text-embedding-3-small 모델은 본 말뭉치에서 관련 있는 문서조차 0.4를 넘기기 어려웠으며, 절대적인 점수보다 상대적인 점수 차이가 더 신뢰할 수 있는 판단 근거가 된다는 것을 배웠습니다.
- **스펙 문서를 문자 그대로 대조하는 과정에서 세 가지 차이를 발견하고 고쳤습니다.** 골든 세트가 `golden.json` 대신 코드에 하드코딩되어 있었고, 비용이 드는 테스트가 `-Peval` 게이팅 없이 기본 `test`에 걸려 있었으며, 실험표가 과제가 제시한 6개 조합 중 일부만 다루고 있었습니다. 세 가지 모두 고치고 나서야 '동작한다'와 '스펙대로 구현되었다'가 서로 다른 확인 항목이라는 것을 체감했습니다.

8.2 향후 개선 아이디어

- **pgvector로 전환하면 좋을 것 같습니다.** 지금은 `SimpleVectorStore`(인메모리 저장소)를 쓰고 있어서 재시작할 때마다 인제스트를 다시 해줘야 하는 게 아쉬웠습니다. 제가 보기엔 pgvector 스타터 하나만 추가해도 `@ConditionalOnMissingBean` 덕분에 코드를 거의 건드리지 않고 전환할 수 있을 것 같아서, 다음에 시간이 되면 직접 붙여 보고 싶습니다.
- **재순위(rerank)를 붙이면 더 좋은 점을 가져갈 수 있을 것 같습니다.** 실험표(6.2절)를 돌려 보니 임계값 하나만으로는 관련 문서와 무관한 문서를 완전히 갈라내기 어렵다는 걸 직접 확인했습니다. 그래서 top-k로 후보를 넉넉히 뽑은 다음, ch08에서 본 `Ranking` 패턴처럼 LLM 기반으로 한 번 더 골라내면 경계에 걸친 질문들을 더 잘 걸러낼 수 있지 않을까 생각합니다.
- **부서·권한 기반 필터도 함께 넣으면 좋을 것 같습니다.** ch07의 `dept` 필터를 보면서, `source` 메타데이터에 접근 권한을 태깅해 `filterExpression`으로 검색 범위 자체를 좁히는 방식이 지금 구현보다 훨씬 안전할 것 같다는 생각이 들었습니다. 사내 문서라는 설정을 감안하면 이 부분은 꼭 필요한 보완이라고 느꼈습니다.

- **Micrometer 지표를 그냥 쌓아만 두지 말고 대시보드로 보면 더 도움이 될 것 같습니다.** 지금은 `/actuator/prometheus`에 값만 쌓이고 있는데, 제 생각에는 `lab2.ask.grounded` 비율을 시계열 그래프로 볼 수 있게 만들면 '요즘 거절이 늘었다' 같은 변화를 코드를 고치지 않고도 먼저 알아챌 수 있을 것 같습니다.